SOFTWARE PROJECT DOCUMENTATION

[OEMS - Online Exam Management System]

Submitted to : Department of CSE, University Of Scholars

Submitted By :

Name	Batch	ID
Md. Rahimujjaman Rahim*	CSE-11	221010020
Md. Shafiqul Islam	CSE-11	221010007
Mst.Humayra Khatun	CSE-11	221010029

Date : November, 2025

Course : Integrated Design Project 1
Course Code : CSE 3180-0113

Supervisor and Instructor : **Eng. Ahsan Arif,**
Head of Department , CSE.

Scan me for the Source Code

Contents

1 Abstraction----------3
2 Introduction----------3
2.1 Motivation---------- 3
2.2 Objectives----------3
3 Problem Statement----------3
4 Related Work / Literature Review----------4
5 System Requirements----------4
5.1 Functional Requirements---------- 5
5.2 Non-Functional Requirements----------5
5.3 Hardware & Software Requirements----------5
6 System Design & Architecture---------- 5
6.1 High-Level Architecture----------6
6.2 Module Breakdown---------- 6
6.3 Database Schema---------- 6
6.4 Sequence / Flow Diagrams----------6
(a) User Registration and Verification Flow----------7
(b) Login and Role-Based Access Flow---------- 7
(c) Quiz Creation and Assignment Flow (Teacher Role)---------- 7
(d) Quiz Attempt and Evaluation Flow (Student Role)---------- 7
7 Features----------8
7.1 Admin Features----------8
7.2 Teacher Features---------- 8
7.3 Student Features----------8
7.4 Exam Management Features---------- 8
7.5 Question Management Features----------9
7.6 Question Set Management Features---------- 9
8 Implementation---------- 9
8.1 Technologies Used---------- 9
8.2 Project Structure---------- 9
📂 src Folder---------- 10
📂 webapp Folder----------10
📂 WEB-INF Folder----------11
8.3 Screenshots----------11
9 Testing & Results----------11
10 Discussion, Challenges & Limitations---------- 12
11 Conclusion & Future Work---------- 12
12 References----------13
13 Appendices----------13

1 Abstraction

This project presents Online Exam Management System, a web-based application built using Java Servlets, JSP, and Oracle database. The system supports role-based access for Admin, Teacher, and Student users and enables functionalities including question bank management, quiz / exam creation, automated evaluation, and result viewing. The system is designed for educational institutions and training platforms to streamline online examination workflows, reduce manual grading, and improve administrative control.

2 Introduction

In modern education and training, managing assessments manually can be tedious, error-prone, and time-consuming. An online quiz/exam management platform can automate the process, enforce integrity via role-based security, and provide instant feedback.

2.1 Motivation

- Teachers spend a lot of time creating, distributing, and grading tests manually.
- Students may face delays in receiving results.
- Students easily do cheating in available online exam system
- Administrators need oversight and control over exams, user management, and permissions.

2.2 Objectives

- To build a web application for managing quizzes/exams online.
- To support multiple roles (Admin, Teacher, Student) with appropriate permissions.
- To enable question bank creation, exam creation, assignment to students, and automatic evaluation.
- To provide intuitive UI and responsive views for different users.
- To integrate email verification for user registration and security.

3 Problem Statement

Educational institutions lack a cohesive system for managing quizzes and exams online, with features like dynamic question sets, secure user access, and automated grading. The absence of such a system leads to manual overhead, human error in evaluation, and delay in result dissemination.

4 Related Work / Literature Review

In recent years, several online quiz and examination management systems have been developed to automate traditional assessment processes. These systems aim to simplify exam administration, question management, evaluation, and result publication. However, most existing solutions either lack flexibility, are difficult to maintain, or are not tailored for institutional customization.

Moodle, an open-source Learning Management System (LMS), provides a comprehensive quiz module with features such as random question generation, time-bound tests, and automated grading. While Moodle is robust and widely used, it is complex to deploy and manage for smaller institutions, requiring dedicated hosting and technical expertise.

Similarly, platforms like Google Forms and Microsoft Forms offer quick quiz creation and automatic grading features. However, these tools are limited in functionality—they do not support multi-role access control, secure login systems, or advanced data management, and are unsuitable for official institutional use due to limited integration and customization options.

Some lightweight web-based quiz management projects built using PHP, Python (Django), or Node.js frameworks exist, but many of them focus only on basic quiz functionalities without role-based authentication, user verification, or detailed result tracking. They also often lack institutional-level features such as question bank management and teacher–student workflow integration.

The *QEMS* (**Quiz Management System V1.2)** addresses these gaps by providing a secure, Java-based web solution built using JSP and Servlets, integrated with an Oracle Database, and featuring role-based authentication for Admin, Teacher, and Student users. It supports email verification, automated evaluation, and centralized database management, making it more scalable and maintainable for real-world institutional use.

5 System Requirements

Before implementing the Quiz Management System, it is essential to define the system requirements to ensure that the software performs efficiently and meets user expectations. The system requirements outline both the functional and non-functional aspects, as well as the hardware and software environment necessary for smooth operation. These specifications help in understanding the scope, performance constraints, and technical dependencies of the project.

5.1 Functional Requirements

- User registration & login with roles (Admin, Teacher, Student)
- Admin can manage users, approve registrations, manage exams
- Teacher can create questions, question sets, exams, assign them
- Student can take quizzes, view results
- Automated scoring and result reporting
- Email-based verification for user sign-up.

5.2 Non-Functional Requirements

- Security (role-based filters, input validation)
- Performance & response time
- Usability / UI responsiveness
- Scalability (moderate load)
- Maintainability (modular code architecture)

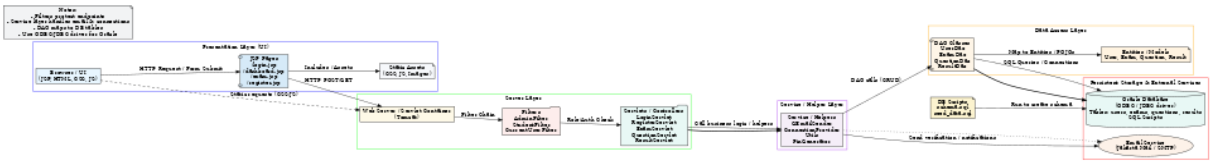
5.3 Hardware & Software Requirements

- Java (21) & JavaEE
- Tomcat 10.1
- Oracle DB (Oracle 21c)
- ODBC11 setup
- Jakarta Mail library
- Web browser for client side
- OS: Windows/Linux etc.

6 System Design & Architecture

The system design and architecture define the overall structure and working mechanism of *QEMS* (**Quiz Management System V1.2)**. It represents how different components of the application—such as the user interface, server logic, database, and communication layers—interact with each other to achieve the desired functionality. A well-structured design ensures scalability, maintainability, and smooth data flow between modules. The architecture follows a layered MVC (Model-View-Controller) pattern, separating the presentation, business logic, and data access layers to improve modularity and system performance.

6.1 High-Level Architecture



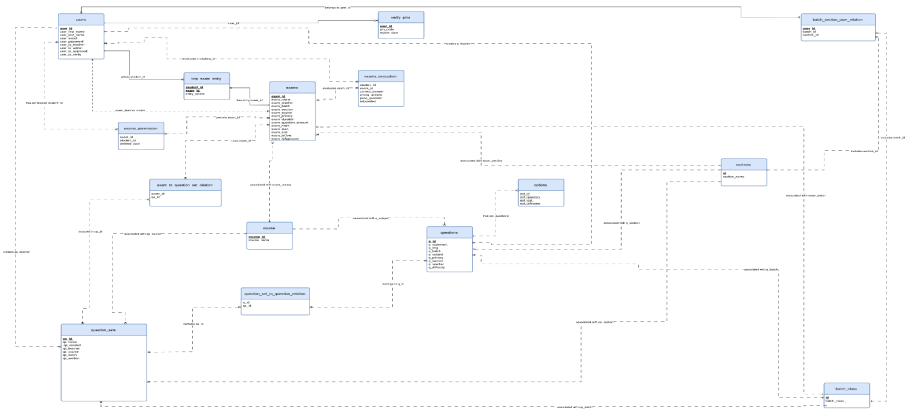
(Block Diagram)

Note: For better view check the provided GitHub repository of the front page

6.2 Module Breakdown

- **Servlets:** handle HTTP requests (Login, CreateExam, EvaluateExam, etc.)
- **DAO:** data operations for users, questions, exams, etc.
- **Entities / POJOs:** mapping DB tables to Java classes (Users, Questions, Options, etc.)
- **Filters:** to check user role and permissions before allowing access to resources (AdminFilter, StudentFilter, TeacherFilter)
- **Helper / Utility Classes:** DB connection provider, email sender, PIN generation, etc.

6.3 Database Schema



(ER Diagram)

Note: For clear view check the svg file from the GitHub repository of the front page

6.4 Sequence / Flow Diagrams

The sequence and flow diagrams represent the logical flow of operations within the Quiz Management System. They illustrate how different users (Admin, Teacher, and Student) interact with the system and how data moves through various modules during typical operations such as registration, quiz creation, and quiz participation.

The system follows a structured and event-driven workflow, ensuring that each operation is validated and executed in a secure and logical order. Below are the main process flows:

(a) User Registration and Verification Flow

1. The user fills out the registration form and submits it.
2. The system generates a **verification PIN** and sends it to the user's email.
3. The user enters the received PIN to verify their account.
4. Upon successful verification, the system activates the user profile.
5. The user can then log in according to their assigned role (Admin, Teacher, or Student).

Flow Summary:

`User → Registration Form → Email Verification → PIN Validation → Account Activation → Login Page`

(b) Login and Role-Based Access Flow

1. The user logs in using their email and password.
2. The **Authentication Servlet** validates the credentials against the database.
3. Based on the user role, the system redirects them to the appropriate dashboard:
   Admin → Admin Dashboard
   Teacher → Teacher Dashboard
   Student → Student Dashboard
4. **Servlet Filters** ensure that only authorized roles can access their respective sections.

Flow Summary:

`User → Login Servlet → Role Validation → Dashboard (Admin/Teacher/Student)`

(c) Quiz Creation and Assignment Flow (Teacher Role)

1. The teacher creates a question bank and saves it in the database.
2. Questions are grouped into sets and assigned to specific batches.
3. The teacher creates a quiz/exam, selecting the question set, time limit, and assigned batch and section.
4. The system stores the quiz details and updates the student dashboard with available quizzes.

Flow Summary:

`Teacher → Question Creation → Question Set → Exam Creation → Database → Student Dashboard`

(d) Quiz Attempt and Evaluation Flow (Student Role)

1. The student logs in and views the available exams.
2. Upon starting a quiz, the system loads the questions dynamically with random order.
3. The student answers and submits the quiz within the time limit.
4. The system automatically evaluates the answers and calculates the score.
5. Results are stored in the database and displayed to the student.

Flow Summary:
`Student → Exam Entry → Answer Submission → Auto Evaluation → Result Display`

7 Features

The **Quiz Management System V1.2** offers a range of features designed to simplify online examination management for all types of users. Each module is tailored to the specific roles of Admin, Teacher, and Student, ensuring secure access and smooth workflow. The system also includes dedicated functionalities for managing exams, questions, and question sets, enabling efficient creation, assignment, and evaluation of quizzes.

7.1 Admin Features

- Manage user accounts and exams (approve, delete).
- Assign additional admin among teachers.
- Monitor ongoing exams, approved exams, permit students to participate in exams.
- View all exams.
- Maintain system security and data integrity.

7.2 Teacher Features

- Create and manage question banks.
- Organize questions into sets based on subjects or difficulty levels.
- Create, schedule, and assign exams to specific student batches.
- Monitor student performance and review results.
- Modify or delete existing question and question sets.
- Modify exams (Cannot modify before 10minutes of starting a exam)

7.3 Student Features

- Register and verify accounts via email PIN verification.
- Log in securely and access available exams.
- Attempt quizzes within the time limit and submit answers.
- View instant results and performance history.
- Reset password information.

7.4 Exam Management Features

- Create exams with configurable time limits and total marks.
- Assign exams to batches or individual students.
- Automatically evaluate submitted answers and calculate scores.
- Display and store results for both teacher and student access.
- Questions will order randomly also the options of the question will order randomly as well.
- Auto Close Exam After the Scheduled time.
- If a student changes the exam tab or minimizes the browser in the middle of exam, he/she will be expelled.
- Student can not change his/her answers after time is over

7.5 Question Management Features

- Add, edit, and delete questions with multiple-choice options.
- Tag questions by category, subject, or difficulty.
- Store questions securely in the central database.
- Reuse questions across multiple exams or question sets.

7.6 Question Set Management Features

- Group multiple questions into organized sets.
- Assign question sets to specific subjects or exams.
- Allow teachers to quickly create balanced and structured quizzes.
- Simplify reusability and management of question banks.

8 Implementation

The implementation phase focuses on transforming the system design into a fully functional application. In this stage, all modules of the Quiz Management System V1.2 or OEMS were developed, integrated, and tested to ensure proper functionality. The system was implemented using Java Servlets, JSP, Oracle Database, and related technologies, following the MVC architecture to maintain modularity and scalability.Deployment Strategy

8.1 Technologies Used

- Java (Servlets & JSP)
- HTML, CSS, JavaScript, Bootstrap
- Oracle DB
- Jakarta Mail for email features

- ODBC11 for DB connectivity
- Web server: Tomcat

8.2 Project Structure

The Quiz Management System V1.2 is organized in a modular structure to ensure clarity, maintainability, and scalability of the application. The project follows the **Model–View–Controller (MVC)** architecture, where Servlets act as controllers, JSP pages handle the presentation layer, and the DAO and entity classes manage data operations and modeling.
The key directories and their purposes are described below:

📂 src Folder

This is the main source directory that contains all backend code files organized into packages. It includes several sub-packages, each serving a specific purpose:

- 📁 ***DAO (Data Access Object):***
  This package contains classes responsible for handling all database operations such as inserting, updating, deleting, and retrieving data.
  Example: `UserDao.java`, `QuestionDao.java`, `ExamDao.java`, etc.

- 📁 ***entities:***
  Contains **Plain Old Java Objects (POJOs)** that represent database tables. Each entity class defines fields corresponding to table columns along with getter and setter methods.
  Example: `User.java`, `Question.java`, `Exam.java`, `QuestionSet.java`.

- 📁 ***filters:***
  Includes Java Servlet filters used for **role-based access control**. These filters ensure that only authorized users (Admin, Teacher, or Student) can access specific pages or functionalities.
  Example: `AdminFilter.java`, `TeacherFilter.java`, `StudentFilter.java`.

- 📁 ***helper:***
  Holds utility classes that support common functions across the project such as database connection handling, email sending, and random PIN generation.
  Example: `ConnectionProvider.java`, `EmailSender.java`, `PinGenerator.java`.

- 📁 ***servlets:***
  Contains the core logic of the application. Each Servlet handles a specific HTTP request, performs business logic, interacts with DAO classes, and forwards responses to the JSP pages.
  Example: `LoginServlet.java`, `RegisterServlet.java`, `CreateExamServlet.java`, `SubmitQuizServlet.java`.

- 📁 ***secret:***
  This package is used to store sensitive configurations such as credentials, tokens, or constants used internally by the system. These files are usually excluded from public deployment for security purposes.

📂 webapp Folder

This directory contains all front-end resources of the project, including JSP pages, HTML templates, and static assets. It serves as the **View** layer of the MVC pattern.

- 📂 ***JSP Pages:***
  Located directly inside the `webapp` folder or within subfolders like `/views`, these files define the structure and content of web pages. They handle the presentation logic for login, registration, dashboards, question creation, quiz participation, and result viewing.
  Example: `login.jsp`, `register.jsp`, `student_dashboard.jsp`, `teacher_dashboard.jsp`, `create_exam.jsp`, `result.jsp`.

- 📂 ***Static Assets:***
  Contain CSS, JavaScript, and image files that enhance the UI and overall user experience.
  Example:
  - `/css/` → styling files (Bootstrap/custom styles)
  - `/js/` → client-side scripts for interactivity and validation
  - `/images/` → logos, icons, or background images used in JSP pages

📂 WEB-INF Folder

This folder contains deployment descriptors and protected resources.

- `web.xml` → defines servlet mappings, filter chains, and access rules.
- JSP pages inside `WEB-INF` are not directly accessible by users for added security.

8.3 Screenshots

Check the given repository. You'll find a folder named screenshots. Also in **README.md** you'll see the screenshots in a manner way.

9 Testing & Results

We test so many cases, some of them are given below with priority and other details. All of them are successfully passed.

TEST CASE ID	TITLE	PRECONDITIONS	TEST STEPS	EXPECTED RESULT	PRIORITY
STU-01	Login with valid credentials	User account exists	Visit login → enter username/password → click Login	Redirect to dashboard	High

STU-02	Login with valid credentials	N/A	Enter wrong/up-case password → Login	Invalid credentials error; stay on login	High
STU-03	Password reset functionality	Valid email registered	Click "Forgot", enter email; receive link; set new password.	Login succeeds with new password	Medium
STU-04	View scheduled exams	Logged in as student	Go to "Scheduled Exams" page	Shows correct exam Hint for the user	High
STU-05	Navigate within exam	Exam in progress	Use **Next, Previous, Skip**, or question list	NAV works as expected; answers saved	High
STU-06	Submit answers	All questions answered	Click **Submit**	Confirmation prompt; result shows scores, correct %	High
STU-07	View exam result	Submission complete	Go to "Show Exam Results"	Shows score,pass/fail,details per question	Medium
STU-08	Logout and session timeout	Use logged in	Wait idle > timeout or explicitly log out	Session invalidated; required re-login	Medium

10 Discussion, Challenges & Limitations

- Challenges: DB connectivity, handling concurrency, email setup, security filters
- Limitations: no advanced analytics, no mobile app, limited scaling, minimal UI polish
- No broad question option.

11 Conclusion & Future Work

The **Quiz Management System V1.2 aka** OEMS successfully automates online quiz management for students, teachers, and admins. It provides secure authentication, efficient exam handling, and instant result generation. Built with **Java Servlets, JSP, and Oracle Database** under the **MVC architecture**, the system ensures reliability, scalability, and ease of maintenance.

Future improvements may include:

- Responsive and modern UI design.

- Broad questions system.
- Detailed performance analytics.
- Cloud database integration.
- Multi-language and AI-based question generation.

12 References

- https://www.oracle.com/technical-resources/articles/javase/servlets-jsp.html
- https://mailtrap.io/blog/jakarta-mail-tutorial/

13 Appendices

Scan the given QR code to visit my github repository to observe full source code and database schema code.

https://github.com/rahimuj570/quize_management_V1.2

THE END